

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY: [FILL IN FACULTY]

FINAL PROJECT REPORT

Course: DevOps

**Designing and Implementing a DevOps, DevSecOps, and
GitOps Process for the WorldMonitor Microservices
Application on Kubernetes**

Instructor: [Fill in instructor name]

No.	Student ID	Full name	Responsibility
1	[Fill in]	[Person 1]	CI/CD, DevSecOps, report
2	[Fill in]	[Person 2]	Kubernetes, GitOps, deployment, testing

Class: [Fill in class code]

Academic year: 2025-2026

Ho Chi Minh City, May 2026

Notes On UIT Report Format

This report follows the common UIT project/thesis submission structure: cover page, summary, table of contents, main chapters, experimental evidence, conclusion, references, and appendices. UIT faculty notices indicate that final reports are commonly submitted as PDF files following the university/faculty template, with additional artifacts such as summary report, presentation slides, software design, and demo video when required. The exact official template may be available through UIT DAA internal forms and can require student login.

Abstract

WorldMonitor is a real-time global intelligence dashboard that aggregates and visualizes geopolitical, military, economic, cyber, climate, aviation, maritime, and infrastructure data. The original project already includes containerization support and several runtime components, but it was not organized as a complete DevOps workflow for Kubernetes.

This project designs and implements a DevOps process for WorldMonitor on Kubernetes. The process includes CI/CD automation with GitHub Actions, DevSecOps checks such as SAST, dependency scanning, IaC scanning, container scanning, and optional DAST, GitOps deployment with Argo CD, and operational monitoring/logging using Prometheus, Grafana, Loki, and Promtail. The deployment target is a local Kubernetes environment such as K3s or Minikube, which is suitable for final project demonstration and repeatable testing.

The result is a working DevOps project repository containing CI/CD workflows, Kubernetes manifests, GitOps configuration, monitoring configuration, and evidence screenshots showing application deployment, health checks, logs, metrics, and self-healing behavior.

Table Of Contents

- 1 Introduction
- 2 Current System Analysis
- 3 Target DevOps Architecture
- 4 CI/CD Pipeline Design
- 5 DevSecOps Integration
- 6 Kubernetes And GitOps Deployment
- 7 Monitoring And Logging
- 8 Testing And Experimental Results
- 9 Work Division

10 Limitations And Future Work

11 Conclusion

12 References

13 Appendices

1. Introduction

1.1 Background

Modern software systems need fast and reliable delivery, repeatable infrastructure provisioning, and continuous security validation. DevOps combines development and operations practices to automate build, test, release, and deployment workflows. DevSecOps extends DevOps by integrating security checks early in the software delivery lifecycle. GitOps uses Git as the source of truth for infrastructure and deployment state, allowing Kubernetes clusters to be reconciled automatically from version-controlled manifests.

1.2 Project Objective

The objective of this project is to design and implement a DevOps pipeline for WorldMonitor, including:

- Automated CI/CD for quality checks, image build, and image publishing.
- DevSecOps checks in the CI/CD workflow.
- Kubernetes deployment for the application services.
- GitOps deployment management using Argo CD.
- Monitoring and logging for runtime observability.
- Deployment and testing evidence for final project evaluation.

1.3 Scope

The project focuses on DevOps enablement and operationalization. It does not refactor the application business logic or split every API domain into a separate microservice. Instead, it maps the existing runtime components into a practical microservices deployment model on Kubernetes.

2. Current System Analysis

2.1 WorldMonitor Overview

WorldMonitor is a TypeScript/Vite single-page application with server-side API handlers and supporting services. It provides a global situational awareness dashboard with map layers, panels, live feeds, and external data integrations.

2.2 Existing Runtime Components

The repository contains these main deployable components:

Component	Description
worldmonitor	Main container serving the SPA and local API sidecar
ais-relay	Relay and data seed service for AIS, RSS/OREF, and selected upstream sources
redis-rest	REST proxy used by the app to access Redis through an Upstash-compatible interface
redis	Stateful cache for data, freshness metadata, and rate-limit state

2.3 Practical Microservices Mapping

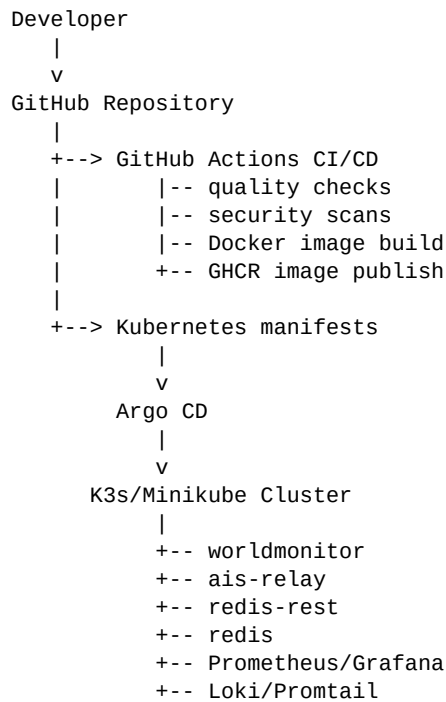
The original application is not a pure domain-by-domain microservices system. For this final project, the architecture is converted into a practical Kubernetes microservices model:

```
User Browser
  |
  v
worldmonitor Service
  |-- serves SPA
  |-- proxies /api/* to local API sidecar
  |
  +--> redis-rest Service --> Redis StatefulSet
  |
  +--> ais-relay Service
```

This model is sufficient to demonstrate DevOps, DevSecOps, GitOps, Kubernetes deployment, monitoring, logging, and operational testing.

3. Target DevOps Architecture

3.1 High-Level Architecture



3.2 Repository Structure Added For The Project

Path	Purpose
<code>.github/workflows/devsecops.yml</code>	Quality gates and security scanning
<code>.github/workflows/container-publish.yml</code>	Build and publish images to GHCR
<code>deploy/kubernetes/</code>	Kubernetes and Argo CD manifests
<code>deploy/monitoring/</code>	Prometheus, Grafana, Loki, Promtail configuration
<code>docs/devops/</code>	Architecture, pipeline, DevSecOps, and report documentation

4. CI/CD Pipeline Design

4.1 Quality Gate Workflow

The `devsecops.yml` workflow runs on pull requests, pushes to `main`, and manual dispatch. It performs:

- Dependency installation with Node.js 22.
- Linting.
- TypeScript type checking.
- API type checking.

- Data/unit tests.
- Sidecar tests.

4.2 Container Publish Workflow

The `container-publish.yml` workflow builds and publishes the following images to GitHub Container Registry:

Image	Source Dockerfile
<code>ghcr.io/<owner>/worldmonitor-app</code>	<code>Dockerfile</code>
<code>ghcr.io/<owner>/worldmonitor-ais-relay</code>	<code>Dockerfile.relay</code>
<code>ghcr.io/<owner>/worldmonitor-redis-rest</code>	<code>docker/Dockerfile.redis-rest</code>

The workflow uses two tag types:

- `sha-<commit-sha>` for reproducible deployment.
- `main-latest` for the latest `main` branch image.

4.3 GitOps Promotion Flow

The preferred promotion flow is:

- 1 Code is merged into `main`.
- 2 GitHub Actions builds and publishes images.
- 3 Kubernetes manifests reference the selected image tag.
- 4 Argo CD detects the Git change.
- 5 Argo CD reconciles the cluster to the desired state.

5. DevSecOps Integration

5.1 SAST

Static application security testing is implemented with Semgrep default rules. The pipeline exports `semgrep.json` as a CI artifact and fails on blocking findings.

5.2 Dependency Scanning

Dependency scanning is implemented with `npm audit` for the root app and supporting packages. The workflow fails on high/critical vulnerabilities when a fix is available.

5.3 IaC And Dockerfile Scanning

Trivy config scanning is used to check Dockerfile and infrastructure configuration issues. This helps detect insecure container or manifest settings before deployment.

5.4 Container Image Scanning

Trivy image scanning is used after building the main application image. The workflow ignores unfixed vulnerabilities and blocks high/critical issues that have a fix path.

5.5 DAST

OWASP ZAP baseline scanning is available through manual workflow dispatch. It requires a deployed URL, so it is run after Person 2 exposes the application in the Kubernetes environment.

6. Kubernetes And GitOps Deployment

6.1 Kubernetes Resources

The Kubernetes deployment includes:

keys.

- Namespace: `worldmonitor`.
- ConfigMap: non-secret runtime configuration.
- Secret template: Redis token/password, relay secret, and optional external API
- Deployments: `worldmonitor`, `ais-relay`, `redis-rest`.
- StatefulSet: `redis`.
- Services: ClusterIP services for each runtime component.
- Ingress: `worldmonitor.local` through Traefik.

6.2 Health Checks And Resources

The application manifests include readiness and liveness probes:

Service	Probe
<code>worldmonitor</code>	HTTP <code>/api/health</code> on port 8080
<code>ais-relay</code>	HTTP <code>/health</code> on port 3004
<code>redis-rest</code>	TCP probe on port 80
<code>redis</code>	<code>redis-cli ping</code> with password

The manifests also include CPU/memory requests and limits, `RuntimeDefault` seccomp profile, disabled privilege escalation, and dropped Linux capabilities.

6.3 Argo CD

The Argo CD Application points to:

```
Repository: https://github.com/Youngboy1609/world-monitor-devops.git  
Revision: main  
Path: deploy/kubernetes  
Namespace: worldmonitor
```

Automated sync is enabled with `prune` and `selfHeal`.

7. Monitoring And Logging

7.1 Prometheus And Grafana

The project includes Helm values for `kube-prometheus-stack`. Grafana is used to visualize:

- CPU usage by pod.
- Memory usage by pod.
- Container restarts.
- Ready pod status.



Figure. Grafana dashboard

7.2 Loki And Promtail

The project includes Helm values for Loki and Promtail. Loki stores application logs, while Promtail collects logs from Kubernetes pods.

- redis
- redis-rest
- two worldmonitor replicas

```
PS E:\world-monitor-devops> kubectl get pods,svc -n worldmonitor
```

NAME	READY	STATUS	RESTARTS	AGE
pod/ais-relay-c99955cb-mhbw	1/1	Running	0	39m
pod/redis-0	1/1	Running	0	39m
pod/redis-rest-8bdc77d5d-qsskg	1/1	Running	0	39m
pod/worldmonitor-54bdb5c9dd-gvk4h	1/1	Running	0	39m
pod/worldmonitor-54bdb5c9dd-x8tqc	1/1	Running	0	30m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/ais-relay	ClusterIP	10.96.137.18	<none>	3004/TCP	39m
service/redis	ClusterIP	10.96.27.214	<none>	6379/TCP	39m
service/redis-rest	ClusterIP	10.96.232.71	<none>	80/TCP	39m
service/worldmonitor	ClusterIP	10.96.65.62	<none>	8080/TCP	39m

```
PS E:\world-monitor-devops>
```

Figure. kubectl pods and services

8.3 Application UI Test

The WorldMonitor UI loads successfully through the deployed service. This confirms that the frontend and backend sidecar are reachable from the browser.

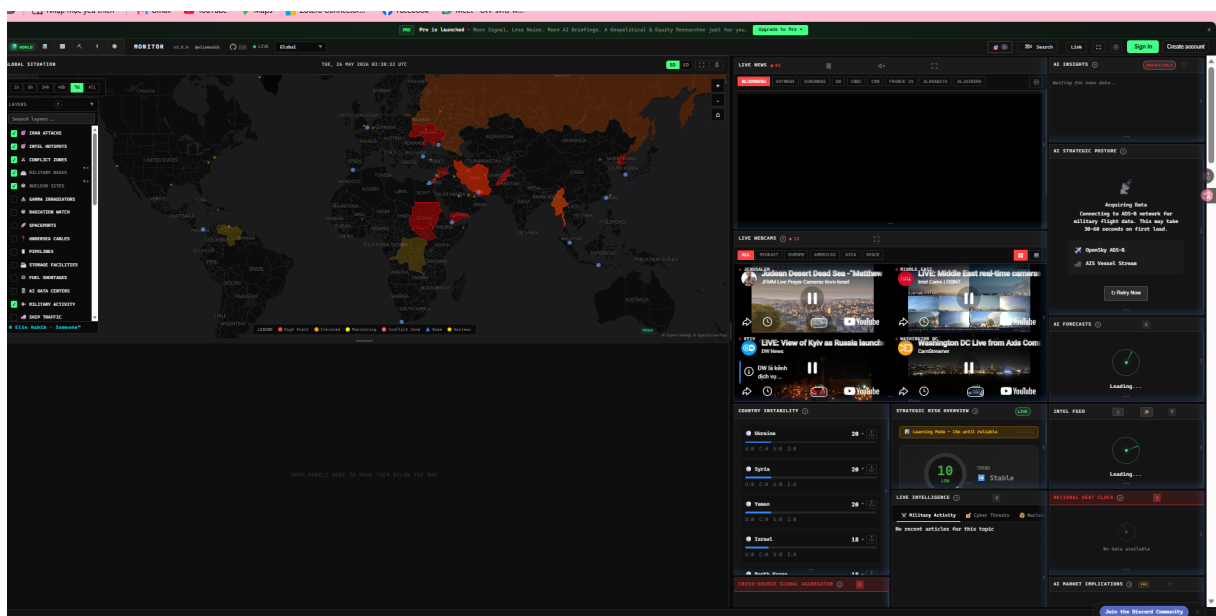


Figure. WorldMonitor UI

8.4 Health Endpoint Test

The `/api/health` endpoint returns HTTP 200. The payload reports overall UNHEALTHY because many live data sources and seed keys are empty, stale, or require optional API keys in the demo environment. This is acceptable for the local DevOps demonstration because the application service itself is reachable, and the health response correctly exposes data freshness problems.

```
PS E:\world-monitor-devops> curl http://localhost:8080/api/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
  Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content         : {
  "status": "UNHEALTHY",
  "summary": {
    "total": 193,
    "ok": 1,
    "warn": 16,
    "onDemandWarn": 28,
    "staleContent": 0,
    "crit": 148
  },
  "checkedAt": "2026-05-26T03:33:20.525Z",
  ...
}
RawContent      : HTTP/1.1 200 OK
                  Transfer-Encoding: chunked
                  Connection: keep-alive
                  Vary: Accept-Encoding,Origin, Accept-Encoding
                  access-control-allow-origin: http://localhost
                  cdn-cache-control: no-store
                  cf-cache...
Forms           : {}
Headers        : {[Transfer-Encoding, chunked], [Connection, keep-alive], [Vary, Accept-Encoding,Origin, Accept-Encoding],
                  [access-control-allow-origin, http://localhost]...}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : mshtml.HTMLDocumentClass
RawContentLength : 19705
```

Figure. Health endpoint

8.5 Logging Test

Loki shows application logs from the `worldmonitor` namespace, including `GET /api/health` requests and health status lines. This confirms that logs are collected from deployed pods.

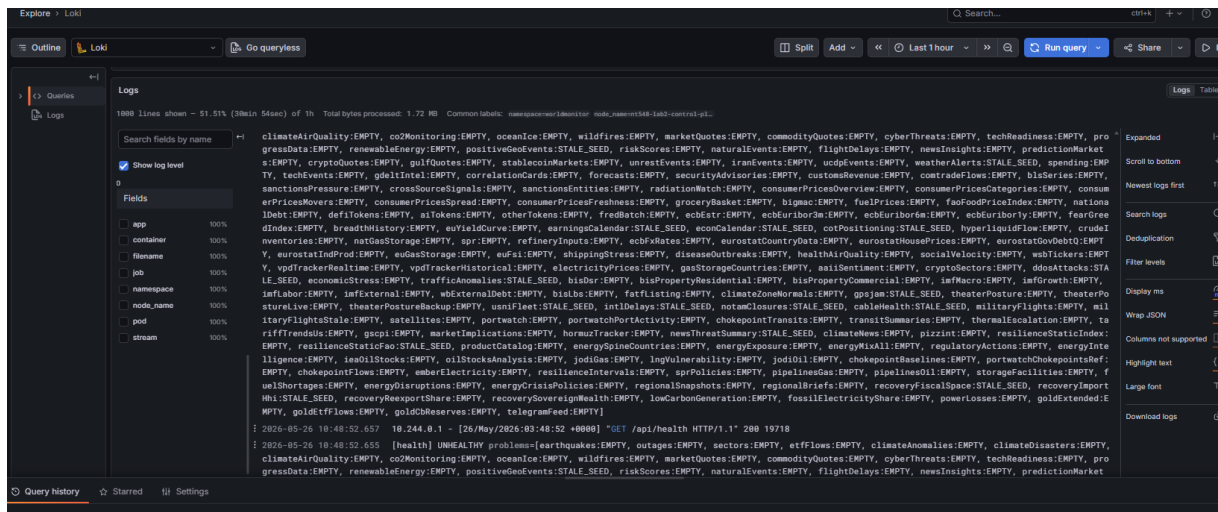


Figure. Loki logs

8.6 Self-Healing Test

The test deletes `worldmonitor` pods using a label selector. Kubernetes recreates the pods, and `kubectl rollout status deployment/worldmonitor -n worldmonitor` completes successfully with two available replicas.

```
PS E:\world-monitor-devops> kubectl delete pod -n worldmonitor -l app.kubernetes.io/name=worldmonitor
pod "worldmonitor-54bdb5c9dd-gvk4h" deleted from worldmonitor namespace
pod "worldmonitor-54bdb5c9dd-x8tqc" deleted from worldmonitor namespace
PS E:\world-monitor-devops> kubectl rollout status deployment/worldmonitor -n worldmonitor --timeout=180s
Waiting for deployment "worldmonitor" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "worldmonitor" rollout to finish: 1 of 2 updated replicas are available...
deployment "worldmonitor" successfully rolled out
PS E:\world-monitor-devops> kubectl get pods -n worldmonitor
NAME                                READY   STATUS    RESTARTS   AGE
ais-relay-c99955cb-mhbwv            1/1     Running   0           73m
redis-0                              1/1     Running   0           73m
redis-rest-8bdc77d5d-qsskg          1/1     Running   0           73m
worldmonitor-54bdb5c9dd-4gl4c        1/1     Running   0           36s
worldmonitor-54bdb5c9dd-c62nj        1/1     Running   0           36s
PS E:\world-monitor-devops>
```

Figure. Self-healing rollout

9. Work Division

9.1 Person 1

Person 1 completed:

- CI/CD workflow design.
- DevSecOps workflow implementation.
- Container publish workflow for GHCR.
- Architecture, pipeline, DevSecOps, and report documentation.

9.2 Person 2

Person 2 completed:

redis.

- Kubernetes manifests for worldmonitor, ais-relay, redis-rest, and
- Argo CD Application for GitOps deployment.
- Prometheus/Grafana and Loki/Promtail configuration.
- Deployment testing screenshots.
- Health check, logging, dashboard, and self-healing evidence.

10. Limitations And Future Work

10.1 Limitations

domain into a separate microservice.

health checks report stale or missing upstream data.

service.

require stricter secret management, TLS, persistent storage policies, and network policies.

- The deployment uses a practical service split instead of refactoring every API
- The local demo uses placeholder or optional API keys, so some WorldMonitor
- DAST requires a deployed URL and should be run manually after exposing the
- The current Kubernetes setup is suitable for local demo; production would

10.2 Future Work

provided by the instructor.

- Add production and staging overlays with Kustomize or Helm.
- Add NetworkPolicy and External Secrets integration.
- Add Argo Rollouts for progressive delivery.
- Add ServiceMonitor resources for application-specific metrics.
- Extend container scanning to all runtime images in a matrix.
- Export the final report to PDF using the official faculty template if

11. Conclusion

The project successfully designs and implements a DevOps workflow for WorldMonitor on Kubernetes. The CI/CD workflows automate quality and security checks, while container publishing prepares images for deployment. Kubernetes manifests define the runtime services, Argo CD manages GitOps synchronization, and the observability stack provides monitoring and logging.

The experimental evidence shows that the application is deployed, synchronized by Argo CD, reachable through the browser, observable through Grafana and Loki, and capable of self-healing after pod deletion. Although the demo environment does not provide all optional API keys and therefore reports some data freshness warnings, the DevOps, DevSecOps, and GitOps objectives are achieved.

12. References

notice 2024-2025." The notice states that each topic submits one PDF report following the thesis presentation template and provides a naming convention.

<<https://se.uit.edu.vn/en/tin-tuc/10-thong-bao-hoc-vu/1809-th%C3%B4ng-b%C3%A1o-n%E1%BB%99p-b%C3%A1o-c%C3%A1o-kh%C3%B3a-lu%E1%BA%ADn-t%E1%BB%91t-nghi%E1%BB%87p-%C4%91%E1%BB%A3t-1-n%C4%83m-h%E1%BB%8Dc-2024-2025.html>>

- 1 Faculty of Software Engineering, UIT. "Graduation thesis report submission

2024-2025." The notice lists report PDF/DOCX, two-page summary, slides, design artifacts, and demo video as submission artifacts when applicable.

<<https://fce.uit.edu.vn/thong-bao-ve-viec-nop-khoa-luan-tot-nghiep-hoc-ky-1-nam-hoc-2024-2025-ban-sau-bao-ve/>>

- 2 Faculty of Computer Engineering, UIT. "Post-defense thesis submission notice

explains that the latest thesis template is available through UIT DAA internal forms and may require login. <<https://http.uit.edu.vn/mau-moi-bieu-mau-khoa-luan-tot-nghiep/>>

- 3 Faculty of Information Systems, UIT. "Updated thesis form notice." The notice

report submission notice." The notice describes project report submission and filename requirements for a course project context. <<https://nc.uit.edu.vn/tin-tuc/thong-bao-hoc-vu/thong-bao-ve-viec-nop-bao-cao-do-an-chuyen-nganh.html>>

- 4 Faculty of Computer Networks and Communications, UIT. "Specialized project

- 5 Kubernetes Documentation. <<https://kubernetes.io/docs/>>

- 6 Argo CD Documentation. <<https://argo-cd.readthedocs.io/>>
- 7 Trivy Documentation. <<https://aquasecurity.github.io/trivy/>>
- 8 Semgrep Documentation. <<https://semgrep.dev/docs/>>
- 9 OWASP ZAP Documentation. <<https://www.zaproxy.org/docs/>>

13. Appendices

Appendix A. Main Commits

Commit	Description
ad9cce4d	Add DevOps project CI and report docs
c25e4e24	Add Kubernetes deployment and observability setup

Appendix B. Screenshots

Evidence	File
Grafana operations dashboard	docs/devops/assets/grafana-dashboard.png
Argo CD Healthy/Synced	docs/devops/assets/argocd-healthy-synced.png
WorldMonitor deployed UI	docs/devops/assets/worldmonitor-ui.png
Kubernetes pods/services	docs/devops/assets/kubectl-pods-services.png
/api/health result	docs/devops/assets/health-endpoint.png
Loki logs	docs/devops/assets/loki-logs.png
Self-healing rollout	docs/devops/assets/self-healing-rollout.png